

Gotta Go Fast

Input file: **standard input**
Output file: **standard output**
Time limit: 1 second
Memory limit: 1024 megabytes

Have you ever played Eggy Party? inf loves playing Eggy Party.

Four eggies are stacked into a four-story eggy tower, with the bottom initially at coordinate 0. From bottom to top, they are sequentially labeled A, B, C, and D, where D is the topmost eggy.

The total length of the track is l , which consists of l segments of length 1. Segment i connects coordinates $i - 1$ and i ($1 \leq i \leq l$).

The terrain of the i -th segment is given by the character S_i . If the eggy tower walks from coordinate $i - 1$ to coordinate i , the cost is:

- Flat ground 0: t_0 seconds;
- Mud pit 1: t_1 seconds;
- Speed pad 2: t_2 seconds.

The eggy tower can perform two types of operations:

- **Normal Advance:** If the current eggy tower is at coordinate $x < l$, it can advance forward to coordinate $x + 1$. The time cost of this advance **depends only on the terrain of segment $x + 1$** .
- **Super Start:** If there is an eggy tower containing j eggies at coordinate x ($j \geq 2$), the bottommost eggy can instantly throw the $j - 1$ eggies above it to coordinate $\min(l, x + d)$, where d is a given positive integer. This operation does not trigger any terrain time cost, i.e., it takes 0 seconds. After the operation, the bottommost eggy at the original position is eliminated, leaving that position empty; a new eggy tower containing $j - 1$ eggies is formed at coordinate $\min(l, x + d)$, and the vertical order of the remaining eggies stays the same.

Please compute the minimum time required for the initially topmost eggy D to reach the destination coordinate l .

Input

The first line contains two positive integers l and d ($1 \leq d < l \leq 2 \times 10^5$).

The second line contains three positive integers t_0, t_1, t_2 ($1 \leq t_2 < t_0 < t_1 \leq 10^9$).

The third line contains a string S of length l , consisting only of the characters 0, 1, and 2.

Output

Output a single integer representing the minimum total time.

Examples

standard input	standard output
10 2 10 20 5 0000000000	40
12 3 10 100 1 111000222000	3
10 2 10 100 1 2220110000	13
8 3 10 100 1 00000000	0

Note

For Example 1: One optimal strategy is to consecutively trigger Super Start 3 times to reach coordinate 6, and then walk through the final 4 flat ground segments. The total time cost is $4 \times 10 = 40$.

For Example 2: One optimal strategy is to first consecutively trigger Super Start twice to reach coordinate 6, walk through 3 speed pads to reach coordinate 9, and finally trigger Super Start again to reach the destination. The total time cost is $1 + 1 + 1 = 3$.

For Example 3: One optimal strategy is to first walk through the first 3 speed pads and 1 flat ground segment to reach coordinate 4, and then consecutively trigger Super Start 3 times to reach the destination. The total time cost is $1 + 1 + 1 + 10 = 13$.

For Example 4: Consecutively trigger Super Start 3 times. The coordinate changes as $0 \rightarrow 3 \rightarrow 6 \rightarrow 8$ (the last time triggers the $\min(8, 6 + 3)$ limit). Walking is not needed throughout the process, and the total time cost is 0.